Лабораторная работа №2
Создание и ведение Git-репозитория

Дисциплина: Технологии разработки программного обеспечения
Студент: Черноусов Олег Андреевич
Группа: еозИСИП - 303
Тема: Система учёта заявок

1. Цель работы
Научиться организовывать разработку проекта с использованием системы контроля версий Git.

2. Ход работы

2.1. Установка и настройка Git.
Git установлен, версия проверена командой `git --version`. Настроены имя и e-mail разработчика:
`git config --global user.name "Chernousov Oleg"`, `git config --global user.email "andreevich.olegg@gmail.com"`.

2.2. Создание репозитория.
Создан публичный репозиторий на GitHub: https://github.com/WEEPINGSCOPE919/Chernousov_Oleg_eozISIP303. Локальный репозиторий инициализирован командой `git init`, привязан к удалённому через `git remote add origin`.

2.3. README.md.
Создан файл `README.md` с названием проекта, назначением, списком функций, стеком и структурой каталогов.

2.4. .gitignore.
Создан файл `.gitignore`, исключающий виртуальное окружение (`venv/`, `.venv/`), кэш Python (`__pycache__/`, `*.pyc`), служебные файлы IDE (`.vscode/`, `.idea/`), локальные базы (`*.db`, `*.sqlite3`) и секреты (`.env`, `secrets.txt`).

2.5. Коммиты.
Выполнено более 5 осмысленных коммитов, отражающих этапы работы: структура проекта, заглушки каталогов, диаграмма Use Case, отчёт по лабе №1, `.gitignore`, обновление README. Сообщения коммитов оформлены по рекомендациям методички (`docs: ...`, `chore: ...`).

2.6. Ветка develop.

Создана ветка `develop` командой `git checkout -b develop`. В неё внесено изменение — файл `docs/develop_notes.md` с заметками по разработке, закоммичено.

2.7. Слияние.
Выполнено переключение на `main` (`git checkout main`) и слияние (`git merge develop`). Конфликтов не возникло, так как изменения затрагивали разные файлы. Изменения отправлены в удалённый репозиторий командами `git push origin main` и `git push origin develop`.

2.8. Работа с нейросетью.
Нейросеть использовалась для объяснения рекомендуемого Git-workflow, последовательности команд, шаблона `.gitignore` и сообщений коммитов. Все команды выполнены самостоятельно и проверены. Совет ИИ скорректирован: предложенная строка `*.log` в `.gitignore` убрана как неактуальная для проекта; сообщение коммита «update» заменено на осмысленное `docs: update README`.

3. AI-журнал

Промпт:
«Ты — опытный DevOps-инженер. Объясни рекомендуемый Git-workflow для учебного проекта: создание репозитория, ветки develop, коммиты, merge. Предложи последовательность команд. Отдельно предложи 5 содержательных сообщений для commit и шаблон .gitignore для Python-проекта. Ограничения: команды должны быть безопасными для новичка, объясни, что делает каждая потенциально опасная команда (merge, reset, push --force).»

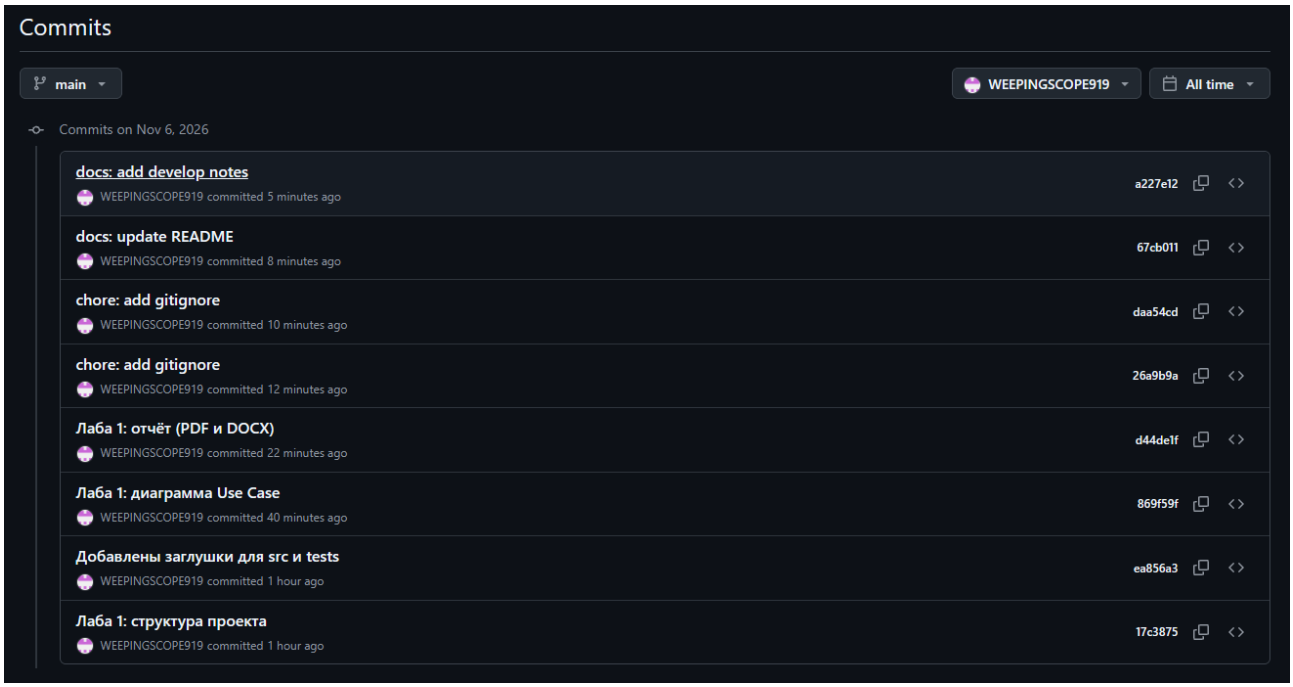
Запись AI-журнала:

1. **Задача:** получить рекомендуемый Git-workflow и последовательность команд для учебного проекта, сообщения коммитов и шаблон `.gitignore`.
2. **Модель:** DEEPSEEK-v3.
3. **Промпт:** Ты — опытный DevOps-инженер. Объясни рекомендуемый Git-workflow для учебного проекта: создание репозитория, ветки develop, коммиты, merge. Предложи последовательность команд. Отдельно предложи 5 содержательных сообщений для commit и шаблон .gitignore для Python-проекта. Ограничения: команды должны быть безопасными для новичка, объясни, что делает каждая потенциально опасная команда (merge, reset, push --force)..
4. **Почему такой промпт:** задал роль, контекст учебного проекта, формат (команды + объяснения) и ограничение «безопасно для новичка» — чтобы не получить совет использовать `push --force` без предупреждения.
5. **Почему такая модель:** выбрана за качество объяснения команд и структурных ответов.

6. **Результат ИИ:** последовательность команд, 5 сообщений коммитов, шаблон `.gitignore`.
7. **Проверка:** выполнил команды сам, проверил `git log`, `git branch -a`, что ветки видны на GitHub; сверил `.gitignore` с реальными папками проекта.
8. **Собственная доработка:** убрал `*.log` из `.gitignore` (в проекте логов нет); заменил сообщение «update» на `docs: update README`.
9. **Вывод:** ИИ хорошо даёт шаблоны, но их надо проверять под свой проект; опасные команды без необходимости не использовать.

4. Вывод
В ходе работы освоено ведение проекта в Git: настройка, создание репозитория, осмысленные коммиты, работа с ветками и слияние. Проект размещён на GitHub, история коммитов и обе ветки (`main`, `develop`) доступны преподавателю. Нейросеть использовалась как инструмент объяснения workflow, все команды выполнены и проверены самостоятельно.

Скриншоты хода выполнения работы:
1.коммиты с гитхаба



2.работа в гитбаше

